

Factory Method

Refactoring en el mundo real — Sistema de reportes con múltiples formatos

Parámetro	Valor
Asignatura	Metodologia de Sistemas II
Modalidad	Trabajo Práctico Individual
Tiempo estimado	2 a 3 horas
Lenguaje	Java, Python, C# o TypeScript
Restricción	Sin librerías externas

¿De qué trata este trabajo?

Esto no es un ejercicio teórico sobre definiciones. Es una simulación de lo que pasa todos los días en proyectos donde la lógica de "qué clase crear" se mezcla con la lógica de "qué hacer con el objeto creado". El resultado son servicios que cada vez que aparece un tipo nuevo de producto hay que tocarlos. Tu trabajo es separar esas dos responsabilidades.

Si entendés bien este patrón, tenés la lógica base para abordar cualquier patrón creacional de la familia GoF por tu cuenta.

Restricción global

No uses librerías externas. Todo se resuelve con el lenguaje base que elijas. El objetivo es demostrar que entendés el patrón, no que encuentraste una librería que lo implementa.

¿Qué está pasando en el proyecto?

Sistema de reportes que genera documentos en distintos formatos. La lógica de "qué clase crear" está mezclada dentro del servicio. Cada vez que aparece un formato nuevo hay que tocar el ReportService, que no debería saber nada de eso.

El código problemático

```
CLASE ReportService:
  MÉTODO generate(data, format_type):
    // decisión de construcción mezclada con lógica de uso
    SI format_type == "pdf":
      report = NUEVO PDFReport()
    SI NO SI format_type == "excel":
```

```
report = NUEVO ExcelReport()
SI NO SI format_type == "csv":
    report = NUEVO CSVReport()
SI NO:
    LANZAR error "Formato no soportado"

// lógica de uso – igual para todos los tipos
report.set_data(data)
report.add_header("Reporte Mensual")
report.add_footer("Generado el " + hoy())
report.render()
RETORNAR report.get_output()
```

El problema de fondo

Construir el objeto y usarlo son responsabilidades distintas viviendo juntas. Si aparece un formato nuevo, hay que tocar el ReportService — que no tiene nada que ver con cómo se crea un reporte. Además, si PDFReport necesita parámetros de construcción distintos, eso también contamina el servicio.

¿Qué hace el patrón Factory Method?

Mueve la lógica de "cuál clase instanciar" a un método o clase separada. El ReportService deja de saber qué clase concreta está usando. Solo sabe que recibe algo que cumple la interfaz Report y trabaja con eso.

Anclaje teórico

Factory Method (también llamado Método fábrica, Constructor virtual) es un patrón de diseño creacional que proporciona una interfaz para crear objetos en una superclase, mientras permite a las subclases alterar el tipo de objetos que se crearán.

La estructura canónica del PDF de teoría identifica cuatro componentes:

- **Producto:** Declara la interfaz común a todos los objetos que puede producir la creadora y sus subclases.
- **Productos Concretos:** Distintas implementaciones de la interfaz de producto.
- **Creador:** Declara el método fábrica que devuelve nuevos objetos de producto. El tipo de retorno coincide con la interfaz de producto.
- **Creadores Concretos:** Sobrescriben el Factory Method base para devolver un tipo diferente de producto.

En este ejercicio, Report es el Producto, PDFReport/ExcelReport/CSVReport/HTMLReport son los Productos Concretos, y ReportFactory concentra la decisión de construcción.

Aclaración importante sobre la implementación

El PDF teórico muestra el Factory Method GoF "canónico" basado en herencia: una clase Creadora abstracta con un método fábrica que las subclases (creadores concretos) sobrescriben (Dialog → WindowsDialog/WebDialog). En este ejercicio usamos una variante

muy difundida en la industria llamada Static Factory o Simple Factory, donde una sola clase con un método estático centraliza la decisión vía switch. Es más liviana cuando solo varía el tipo de producto y no el comportamiento del creador. Si querés acercarte más al GoF estricto, podés implementar PdfReportService, ExcelReportService, etc. como subclases de un ReportService abstracto que defina createReport() — esa decisión también es válida y se evalúa positivamente.

Diagrama UML — Antes vs Después

(Incluí en tu entrega un diagrama UML de la situación inicial y otro de la solución refactorizada.)

Pseudocódigo de la solución

1. La interfaz que todos los reportes cumplen

```
INTERFAZ Report:
  MÉTODO set_data(data)
  MÉTODO add_header(text)
  MÉTODO add_footer(text)
  MÉTODO render()
  MÉTODO get_output() -> String
```

2. Las clases concretas

```
CLASE PDFReport    IMPLEMENTA Report: // lógica de generación PDF
CLASE ExcelReport  IMPLEMENTA Report: // lógica de generación Excel
CLASE CSVReport    IMPLEMENTA Report: // lógica de generación CSV
CLASE HTMLReport   IMPLEMENTA Report: // nuevo – sin tocar nada más
```

3. La fábrica concentra la decisión de construcción

```
CLASE ReportFactory:
  MÉTODO ESTÁTICO create(format_type) -> Report:
    SI format_type == "pdf":    RETORNAR NUEVO PDFReport()
    SI format_type == "excel":  RETORNAR NUEVO ExcelReport()
    SI format_type == "csv":    RETORNAR NUEVO CSVReport()
    SI format_type == "html":   RETORNAR NUEVO HTMLReport()
    SI NO: LANZAR error "Formato no soportado"
```

4. El servicio ahora es agnóstico al formato

```
CLASE ReportService:
  MÉTODO generate(data, format_type):
    report = ReportFactory.create(format_type) // no sabe qué tipo es
    report.set_data(data)
    report.add_header("Reporte Mensual")
    report.add_footer("Generado el " + hoy())
    report.render()
    RETORNAR report.get_output()
```

```
// Agregar HTML: solo ReportFactory cambia. ReportService no se toca.
servicio = NUEVO ReportService()
servicio.generate(datos, "html")
```

¿Por qué Factory Method y no Abstract Factory?

Abstract Factory crea familias de objetos relacionados que deben construirse juntos. Ejemplo: botón + input + checkbox que comparten un mismo tema visual — todos de la misma familia. Acá solo hay un tipo de objeto (Report) en distintas variantes. Factory Method es suficiente. Incluí este razonamiento como comentario en tu código — es parte de la evaluación.

Tu misión

- Refactorizá el sistema con Factory Method.
- Agregá HTMLReport como cuarto formato sin tocar ReportService.
- Justificá en un comentario por qué usaste Factory Method y no Abstract Factory.
- Si elegís la variante GoF estricta (con jerarquía de creadores), explicá por qué; si elegís Static Factory, también.

Restricciones

La firma pública generate(data, format_type) no puede cambiar. ReportService no puede importar ni instanciar ninguna clase concreta de reporte directamente.

¿Qué espero ver en tu entrega?

Qué evalúo	Lo que estoy mirando
Diagrama UML	Antes y después. La jerarquía de productos y dónde quedó la decisión de construcción.
Antes / Después	El código original junto al refactorizado. Sin el contraste no sé si entendiste qué estaba mal.
Justificación	Máximo 10 líneas explicando qué problema específico resolvió el patrón en este ejercicio. No la definición del libro.
Legibilidad	El código refactorizado debe entenderse sin explicaciones.
Factory vs Abstract Factory	El comentario sobre la elección. Quiero ver que la decisión fue consciente, no que fue la primera que salió.
Variante elegida	Si fuiste por GoF estricto o Static Factory — y por qué.

Qué evalúo	Lo que estoy mirando
Restricciones	Todas respetadas. Librería externa = ejercicio no cuenta, aunque el código sea brillante.

Última cosa

Si tu solución no encaja del todo con el patrón "canónico" del libro, explicá por qué la elegiste así. Eso me dice más sobre cómo pensás que si copiaste la implementación de un tutorial. El objetivo es que salgas de acá sabiendo cuándo y por qué aplicar cada patrón.